OpsPilot AI - Sprint 1: Alert Intake

Goal: Allow monitoring tools to send structured alerts to OpsPilot AI through an API.

What We Built

- Created **POST /alerts**.
- Defined required alert fields and allowed values.
- Added automatic validation using Pydantic.
- Generated a unique alert ID and UTC timestamp.
- Tested successfully in Swagger UI with **201 Created**.

Project Flow

```
Monitoring tool -> POST /alerts -> Validate data -> Generate ID and timestamp -> Return acknowledgement
```

Main Code Added

```
from datetime import datetime, timezone
from typing import Literal
from uuid import uuid4
from pydantic import BaseModel, Field

class Alert(BaseModel):
  alert_type: Literal["HIGH_CPU", "HIGH_MEMORY", "DISK_FULL",
    "SERVICE_DOWN", "HTTP_5XX_SPIKE"]
  server: str = Field(min_length=2)
  value: float
  threshold: float
  severity: Literal["low", "medium", "high", "critical"]

@app.post("/alerts", status_code=201)
def receive_alert(alert: Alert):
  return {
    "alert_id": str(uuid4()),
    "status": "received",
    "received_at": datetime.now(timezone.utc).isoformat(),
    "alert": alert
  }
```

Alert Fields

Field	Purpose
alert_type	Type of issue, such as HIGH_CPU.
server	Affected machine or service.
value	Current measured value.
threshold	Limit that triggered the alert.
severity	low, medium, high, or critical.

Key Concepts

Concept	Meaning
BaseModel	Defines the alert structure and validates incoming JSON.
Literal	Restricts a field to approved values.
Field	Adds validation rules such as minimum length.
POST	Sends new data to the backend.
UUID	Unique identifier used to track each alert.
UTC	Standard time format that avoids timezone confusion.

Sample Request

```
{
  "alert_type": "HIGH_CPU",
  "server": "web-server-01",
  "value": 95,
  "threshold": 80,
  "severity": "critical"
}
```

Sample Response

```
{
  "alert_id": "generated-unique-id",
  "status": "received",
  "received_at": "UTC date and time",
  "alert": { submitted alert data }
}
```

How Validation Works

FastAPI and Pydantic validate the request **before** receive_alert() runs. Requests are rejected when a field is missing, a value is unsupported, the server name is too short, or text is sent where a number is expected.

Important Status Codes

Status	Meaning
201 Created	Alert accepted successfully.
422 Validation Error	Request body does not match the Alert model.
404 Not Found	Endpoint does not exist.
500 Internal Server Error	Backend failed during processing.

Why Sprint 1 Matters

Alert intake is the entry point of OpsPilot AI. Monitoring tools such as Nagios, New Relic, Prometheus, or CloudWatch can later send alerts in this standard format. Future sprints will investigate the alert, find the probable root cause, and recommend remediation.

Judge Explanation

"We developed a validated alert-ingestion API. Monitoring tools can send standardised operational alerts, and the platform assigns each alert a unique ID and UTC timestamp. Strict validation ensures only complete and supported data enters the investigation workflow."

Quick Judge and Interview Questions

Q: Why POST instead of GET?
A: The client is sending a new alert to the system.

Q: Why Pydantic?
A: It automatically validates incoming data and creates the API schema.

Q: Why Literal?
A: It prevents inconsistent alert types and severity values.

Q: Why UUID?
A: It uniquely identifies and tracks each alert.

Q: Why UTC?
A: It avoids confusion across different time zones.

Q: What happens with invalid data?
A: FastAPI rejects it with a 422 validation response.

Sprint 1 Summary

OpsPilot AI can now receive, validate, identify, timestamp, and acknowledge structured monitoring alerts.